2. Inject an `InputDestination` bean representing the input binding `packlabel-in-0` (by default, since it's the only one).
3. Inject an `OutputDestination` bean representing the output binding `packlabel-out-0` (by default, since it's the only one).

The data flow is based on `Message` objects (from the `org.springframework.messaging` package). The framework handles type conversion for you transparently when running the application. However, in this type of test, you need to provide `Message` objects explicitly. You can use `MessageBuilder` to create the input message, and use the `ObjectMapper` utility to perform the type conversion from the binary format used for storing message payloads in a broker.

Listing 10.16 Testing the integration with external messaging systems

```
package com.polarbookshop.dispatcherservice;

import java.io.IOException;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.cloud.stream.binder.test.InputDestination;
import org.springframework.cloud.stream.binder.test.OutputDestination;
import org.springframework.cloud.stream.binder.test.
➥TestChannelBinderConfiguration;
import org.springframework.context.annotation.Import;
import org.springframework.integration.support.MessageBuilder;
import org.springframework.messaging.Message;
import static org.assertj.core.api.Assertions.assertThat;

@SpringBootTest
@Import(TestChannelBinderConfiguration.class)                    ❶
class FunctionsStreamIntegrationTests {

  @Autowired
  private InputDestination input;                                ❷
```